

COLECCIÓN RUST PÍXEL A PÍXEL

Cuaderno 3 de Rust

Modelando el Mundo Real

Recomendado a partir de los 17-18 años



Prototipo Editorial

Laboratorio de Código Instantáneo

Índice General

1. Introducción: De variables sueltas a objetos con sentido	1
1.1. El problema de los datos huérfanos	1
1.2. Pensar en «Objetos con Sentido»	2
2. Estructuras de Datos (struct)	3
2.1. El plano arquitectónico: Definición de Struct	3
2.2. Construyendo el objeto: Instanciación y acceso a campos	3
2.3. La mutabilidad en bloque: Modificar datos en una estructura	4
2.4. Estructuras alternativas: Tuple Structs y Unit Structs	5
3. Añadiendo Superpoderes: Métodos y Bloques impl	7
3.1. ¿Qué es un bloque impl? Separando los datos del comportamiento	7
3.2. El espejo de la estructura: Comprendiendo self, &self y &mut self	7
3.3. Funciones asociadas: Constructores personalizados (new)	11
3.4. 🎮 Proyecto Práctico I: El simulador de inventario de una tienda de videojuegos .	11
4. Enumerados (enum): La Joya de la Corona de Rust	16
4.1. Enumeraciones con datos asociados	16
4.2. El tipo Option<T>: La solución definitiva al peligro del valor «Nulo»	17
4.3. La instrucción match	19
5. Control de Flujo Avanzado: Coincidencia de Patrones (match)	21
5.1. La potencia de match: Desestructuración exhaustiva de datos	21
5.2. El comodín de seguridad: El patrón por defecto o guión bajo _	22
5.3. El atajo elegante: Control de flujo simplificado con if let	23
5.4. Proyecto Práctico II: Mini-RPG de texto basado en turnos	24
6. El mapa definitivo de la seguridad: Option y Result	30
6.1. La caja de la presencia o ausencia: Option<T>	30
6.2. La caja del éxito o el fracaso: Result<T, E>	30
6.3. 🛠 El Arsenal de Extracción: ¿Cómo destapamos las cajas?	31
6.3.1. El cirujano del código: La sentencia match	32
6.4. El filtro rápido: La estructura if let	35
6.5. El plan B: El método unwrap_or()	35
6.6. Los botones de autodestrucción: unwrap() y expect()	35
6.7. El pase de oro para profesionales: El operador de propagación ?	36
6.8. Resumen para llevar en la mochila:	38
7. Limpieza y Orden: Organización en Módulos (mod)	39
7.1. ¿Por qué un solo archivo main.rs ya no basta?	39
7.2. Creación de módulos locales y la palabra clave pub	39
7.3. Estructura multi-archivo: Separando la lógica en ficheros independientes	40
7.4. Proyecto Práctico III: Validador robusto de datos de usuario en producción	42
8. Ejercicios Resueltos del Cuaderno 3	45
8.1. ENUNCIADOS	45
8.1.1. Ejercicio 1: Crea tu propia Struct y Función Asociada (Constructor)	45
8.1.2. Ejercicio 2: Métodos Mutables (&mut self)	45
8.1.3. Ejercicio 3: Enumeraciones con Datos Asociados	45

8.1.4. Ejercicio 4: Extrayendo Datos con <code>match</code>	45
8.1.5. Ejercicio 5: El patrón por defecto <code>_</code> e <code>if let</code> con <code>option</code>	45
8.2. SOLUCIONES	45
8.2.1. Ejercicio 1: Crea tu propia <code>Struct</code> y Función Asociada	45
8.2.2. Ejercicio 2: Métodos Mutables (<code>&mut self</code>)	46
8.2.3. Ejercicio 3: Enumeraciones con Datos Asociados	46
8.2.4. Ejercicio 4: Extrayendo Datos con <code>match</code>	47
8.2.5. Ejercicio 5: El patrón por defecto <code>_</code> e <code>if let</code> con <code>option</code>	47

1. Introducción: De variables sueltas a objetos con sentido

Hasta este punto de tu aventura con **Rust**, has aprendido a manejar la información utilizando variables individuales. Si necesitabas almacenar la vida o salud de un jugador, creabas un **let salud = 100**; Si querías su nombre, añadías un **let nombre = String::from("Arturo")**;

Este enfoque funciona perfectamente para programas pequeños o para programas matemáticos aislados. Sin embargo, el software del mundo real no funciona con datos desconectados; funciona con **conceptos integrados**.

1.1. El problema de los datos huérfanos

Imagina que estás diseñando el código de un videojuego multijugador. Cada jugador en el mapa necesita un nombre, una posición en el eje X, una posición en el eje Y y un estado de energía. Usando únicamente lo aprendido en los cuadernos anteriores, tu código inicial se vería más o menos así:

```
fn main() {  
    // Datos del Jugador 1  
    let j1_nombre = String::from("Falcon_Retro");  
    let j1_pos_x = 10.5;  
    let j1_pos_y = 20.0;  
    let j1_energia = 100;  
  
    // Datos del Jugador 2  
    let j2_nombre = String::from("Rustaceo99");  
    let j2_pos_x = -5.0;  
    let j2_pos_y = 14.2;  
    let j2_energia = 85;  
  
    // Si tuviéramos 50 jugadores, ¡necesitaríamos  
    // 200 variables sueltas!  
}
```

Para nuestra mente humana, **j1_nombre** y **j1_energia** están íntimamente relacionados: pertenecen al mismo jugador. Pero para el compilador de Rust, estas variables son completamente desconocidas entre sí. Son solo datos flotando de forma independiente en la memoria de la computadora.

Esto acarrea tres problemas graves a medida que el proyecto crece:

1. **Fragilidad:** Si pasas la posición (x, y) de un jugador a una función para calcular el movimiento, podrías equivocarte por accidente y pasar la coordenada X del Jugador 1 y la coordenada Y del Jugador 2. El compilador no protestará, porque ambos son números decimales, pero provocarás un bug (error) catastrófico en el juego.
2. **Falta de cohesión:** Las funciones que modifican las propiedades del jugador requieren que les pases una lista kilométrica de argumentos individuales en los parámetros de la función: todas las variables del jugador.

3. **Escalabilidad imposible:** Crear dinámicamente nuevos jugadores o eliminarlos cuando se desconecten de la partida se vuelve una tarea titánica si sus propiedades (variables) están dispersas.

1.2. Pensar en «Objetos con Sentido»

Modelar el mundo real significa dejar de pensar en variables aisladas, y empezar a pensar en **entidades completas**.

En lugar de ver cuatro variables huérfanas, queremos que nuestro código entienda el concepto unificado de un **Jugador**. Queremos empaquetar esos datos dispersos dentro de un contenedor único. De esta forma, si movemos (eliminamos) al jugador, movemos el paquete completo; si el jugador se cura, alteramos una propiedad interna de ese paquete.

En este cuaderno aprenderás a diseñar tus propios bloques de construcción para dotar a Rust de **tipos personalizados** que no existen en el lenguaje por defecto. Pasaremos de gestionar números y letras inconexos a modelar **Inventarios, Héroes, Monstruos, etc.**

Damos la bienvenida a la programación estructurada. Es hora de levantar los planos de nuestro código.

2. Estructuras de Datos (`struct`)

📌 **Nota:** En el apartado: 9.6. del Cuaderno 1, "El struct Persona y los superpoderes automáticos", mencionamos muy de pasada las estructuras (`struct`) como herramientas para organizar la información. En este cuaderno las estudiaremos a fondo.

Imagina que estás programando un videojuego y quieres guardar la información de un jugador. Hasta ahora —como ya hemos mencionado en la introducción—, usando lo aprendido en cuadernos anteriores tendrías que crear variables sueltas:

```
let jugador_nombre = String::from("Falcon_Retro");
let jugador_salud = 100;
let jugador_nivel = 5;
```

Este enfoque tiene un problema grave: para el compilador de Rust, estas tres variables no tienen ninguna relación entre sí. Si tienes 50 jugadores en una partida, gestionar cientos de variables sueltas se volvería una pesadilla. Aquí es donde entran las **estructuras (`struct`)**, que nos permiten agrupar variables de diferentes tipos bajo un mismo nombre con sentido.

2.1. El plano arquitectónico: Definición de Struct

Una estructura funciona exactamente como el plano de una casa. No es la casa en sí misma, sino el diseño que define qué elementos la compondrán.

Para definirla, usamos la palabra clave **`struct`**, seguida del nombre en mayúscula (**Camel-Case**) y abrimos llaves. Dentro, definimos los campos indicando su nombre y su tipo de dato.

📌 **Nota:** CamelCase se aplica a nombres compuestos. Por ejemplo, si el nombre de la estructura fueran dos palabras, como "jugador lateral", el nombre resultante de la estructura del jugador sería JugadorLateral. Se unen las palabras poniendo la inicial de cada una en mayúsculas.

```
// Definimos el plano de lo que es un "Personaje" en nuestro código
struct Personaje {
    nombre: String,
    salud: u32,
    nivel: u16,
    es_activo: bool,
}
```

2.2. Construyendo el objeto: Instanciación y acceso a campos

Una vez que tenemos el plano (`struct`), podemos «construir o crear» personajes reales en memoria. A este proceso lo llamamos instanciación. Para leer los datos de un campo específico, utilizamos el operador punto (`.`).

📌 **Nota:** La definición de las estructuras en el código se colocan fuera del main, arriba o abajo del mismo.

📄 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **struct_personaje.rs**

```
// Definimos el plano de lo que es un "Personaje" en nuestro código
struct Personaje {
    nombre: String,
    salud: u32,
    nivel: u16,
    es_activo: bool,
}

fn main() {
    // Instanciamos (creamos) el personaje basándonos en el plano
    let héroe = Personaje {
        nombre: String::from("Aragorn"),
        salud: 100,
        nivel: 1,
        es_activo: true,
    };

    // Accedemos a sus datos individuales usando el punto
    println!("¡Bienvenido al mundo, {}!", héroe.nombre);
    println!("Tu salud inicial es de {} puntos y eres nivel {}.", héroe.salud,
héroe.nivel);
}
```

Comprueba en las instrucciones de salida **println!** cómo se sustituyen las propiedades o campos de la variable **héroe** en los marcadores {}.

2.3. La mutabilidad en bloque: Modificar datos en una estructura

¿Qué pasa si nuestro personaje recibe un golpe y su salud baja? En Rust, la mutabilidad afecta a toda la estructura por igual. No puedes hacer que solo un campo sea mutable; toda la instancia debe declararse con **mut**.

📄 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **struct_personaje_mutable.rs**

```
struct Personaje {
    nombre: String,
    salud: u32,
    nivel: u16,
    es_activo: bool,
}

fn main() {
    // Declaramos la instancia como mutable usando 'mut'
    let mut enemigo = Personaje {
```

```

    nombre: String::from("Orco Gruñón"),
    salud: 80,
    nivel: 2,
    es_activo: true,
};

println!("El {} bloquea el camino con {} de vida.", enemigo.nombre,
enemigo.salud);

// ¡El jugador ataca! Modificamos los campos internos
enemigo.salud = 50; // El enemigo pierde 30 de salud
enemigo.nivel = 3; // El enemigo se enfurece y sube 1 de nivel

println!("Tras el impacto, el {} tiene {} de vida y nivel {}.",
enemigo.nombre, enemigo.salud, enemigo.nivel);
}

```

Comprueba el resultado de las instrucciones **println!** antes y después de modificar las propiedades / campos de la variable **enemigo**.

2.4. Estructuras alternativas: Tuple Structs y Unit Structs

A veces no necesitas ponerle nombre a cada campo porque su significado es evidente, o simplemente necesitas un tipo sin datos para representar un concepto. Rust nos da dos herramientas secundarias muy útiles:

1. **Tuple Structs:** Son estructuras que tienen tipo pero no nombres en sus campos. Ideales para coordenadas espaciales o colores.
2. **Unit Structs:** Estructuras completamente vacías. Son útiles cuando más adelante veamos *Traits* (rasgos/interfaces) donde un tipo necesita demostrar un comportamiento pero no requiere almacenar datos.

📌 Nota: Recuerda que una tupla es un tipo de dato compuesto formado por un conjunto de datos que pueden ser de distinto tipo, separados por comas y encerrados entre paréntesis.

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **tupla_struct.rs**

```

// Una estructura de tupla para almacenar coordenadas en 3D (X, Y, Z)
// Los campos no tienen nombre (se sobreentienden)
struct Posicion3D(f32, f32, f32);

// Una estructura unitaria para marcar un estado o evento
// Veremos más adelante como utilizarla
struct FinDelJuego;

fn main() {
    // Instanciamos la Tuple Struct
    let origen = Posicion3D(0.0, 15.2, -3.4);
}

```

```
// Para acceder a sus campos, usamos índices numéricos
// empezando en 0 como en las tuplas normales
println!("El jugador está en la altura Y: {}", origen.1);
}
```

Verifica el resultado de la instrucción **println!**.

3. Añadiendo Superpoderes: Métodos y Bloques `impl`

En el apartado anterior aprendimos a agrupar variables dentro de una estructura (`struct`). Sin embargo, hasta ahora, si queríamos modificar un personaje o calcular algo con sus datos, teníamos que programar funciones en el `main` externas a la estructura.

En Rust, podemos asociar funciones y métodos o comportamientos **directamente** a nuestras estructuras. Para ello utilizamos el bloque de implementación: `impl`. La diferencia entre funciones asociadas y métodos la veremos más adelante pero señalemos ahora que un método es una función especial.

3.1. ¿Qué es un bloque `impl`? Separando los datos del comportamiento

A diferencia de otros lenguajes de programación donde los datos y las funciones se mezclan dentro de una «Clase», **Rust prefiere mantenerlos separados pero conectados**. En la `struct` defines qué datos tiene el objeto, y en el `impl` defines qué sabe hacer dicho objeto.

```
// 1. El contenedor de datos (El plano)
struct NaveEspacial {
    nombre: String,
    escudo: u32,
    municion: u32,
}

// 2. El bloque de comportamiento (Los superpoderes)
impl NaveEspacial {
    // Aquí dentro programaremos todas las funciones
    // exclusivas de la NaveEspacial
}
```

Arriba podemos ver dos bloques de código. El bloque `struct` en el que se definen las propiedades de la estructura `NaveEspacial` y el bloque `impl` que todavía está vacío pero es donde programaremos **las funciones y los métodos** de la estructura `NaveEspacial`.

3.2. El espejo de la estructura: Comprendiendo `self`, `&self` y `&mut self`

Para que una función dentro de un `impl` se considere un **método**, debe recibir como primer parámetro la palabra clave `self` o una de sus variantes como veremos luego. `self` representa el **objeto** o **instancia** de la estructura que está ejecutando el **método** o **acción**.

Hemos visto que las estructuras (`struct`) sirven para crear objetos del tipo de la estructura. Cada objeto creado se denomina **instancia** y el proceso de creación se denomina **instanciación**.

Una vez que tenemos creado un objeto o instancia de la estructura, podremos ejecutar sus métodos con el operador punto (`.`).

❗ Nota: Supongamos que hemos creado un objeto o instancia denominado **halcon** de una estructura denominada **Pajaro** y que la estructura tiene un método denominado **volar()**. Bien, pues entonces podemos ejecutar el método **volar()** del objeto **halcon**

mediante la expresión `halcon.volar()`. Aquí, la instancia u objeto es `halcon` y el método o acción es `volar()`. **halcon** es el **self** que veremos a continuación. En este apartado construiremos un escenario similar al descrito.

Los métodos operan sobre los datos del objeto (struct). Dependiendo de lo que queramos hacer con los datos, utilizaremos una de las tres siguientes variantes como primer parámetro en un método:

- **&self (Lectura):** Solo queremos leer los datos del objeto (por ejemplo, mostrar el estado en pantalla). Es la más común.
- **&mut self (Modificación):** Necesitamos alterar los datos internos del objeto (por ejemplo, cuando la nave recibe un disparo o gasta munición).
- **self (Consumo):** El método toma el control total del objeto (la propiedad) y lo destruye al terminar el método (se usa poco, por ejemplo, para transformar un objeto en otra cosa).

Veámoslo en acción con la nave:

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **impl_metodos.rs**

La explicación del programa está después del listado.

```
// 1. El contenedor de datos (El plano)
struct NaveEspacial {
    nombre: String,
    escudo: u32,
    municion: u32,
}

impl NaveEspacial {
    // Método de LECTURA (&self): No modifica nada, solo muestra información
    fn reportar_estado(&self) {
        println!("🚢 [{}] Escudo al {}% | Munición: {} torpedos.", self.nombre,
self.escudo, self.municion);
    }

    // Método de MODIFICACIÓN (&mut self): Altera las variables internas
    fn recibir_disparo(&mut self, daño: u32) {
        if daño >= self.escudo {
            self.escudo = 0;
            println!("💣 ¡AVISO! El escudo de la nave {} se ha destruido.",
self.nombre);
        } else {
            self.escudo -= daño;
            println!("💣 ¡Impacto! El escudo absorbió el daño.");
        }
    }
}

fn main() {
    // Es obligatorio usar 'mut' para poder llamar a métodos que usen &mut self
```

```
let mut mi_caza = NaveEspacial {
    nombre: String::from("Halcón Milenario"),
    escudo: 100,
    municion: 10,
};

mi_caza.reportar_estado(); // Llama al método de lectura
mi_caza.recibir_disparo(40); // Llama al método de modificación
mi_caza.reportar_estado(); // Volvemos a leer para comprobar los cambios
}
```

Explicación del programa

En primer lugar definimos nuestra estructura con tres campos.

El bloque **struct**

```
struct NaveEspacial {
    nombre: String,
    escudo: u32,
    municion: u32,
}
```

Los objetos que creamos con esta estructura tendrán los métodos definidos en el bloque impl.

El bloque **impl**

```
impl NaveEspacial {
    // Método de LECTURA (&self): No modifica nada, solo muestra información
    fn reportar_estado(&self) {
        println!("🚢 [{}] Escudo al {}% | Munición: {} torpedos.", self.nombre,
self.escudo, self.municion);
    }

    // Método de MODIFICACIÓN (&mut self): Altera las variables internas
    fn recibir_disparo(&mut self, daño: u32) {
        if daño >= self.escudo {
            self.escudo = 0;
            println!("💣 ¡AVISO! El escudo de la nave {} se ha destruido.",
self.nombre);
        } else {
            self.escudo -= daño;
            println!("💣 ¡Impacto! El escudo absorbió el daño.");
        }
    }
}
```

Como podemos ver hemos definido dos métodos, cuyas cabeceras son:

- fn reportar_estado(&self)
- fn recibir_disparo(&mut self, daño: u32)

Observa que en la función main se crea el objeto mutable **mi_caza** con la instrucción:

```
let mut mi_caza = NaveEspacial {
    nombre: String::from("Halcón Milenario"),
    escudo: 100,
    municion: 10,
};
```

La forma en que el objeto que hemos creado ejecuta los métodos anteriores es la siguiente (ver en el main):

1) `mi_caza.reportar_estado()`

Como vemos en la cabecera del método **fn reportar_estado(&self)**, recibe como primer y único parámetro un **prestamo** (o referencia) **&self**. **self** representa el objeto que está llamando al método con el punto, es decir, **mi_caza**. Esto permite al método acceder a las propiedades del objeto en el interior del método (ejemplo **self.nombre**). Al haber recibido un préstamo **&self**, cuando el método termina de hacer su trabajo el objeto **mi_caza** sigue activo, no se destruye.

2) `mi_caza.recibir_disparo(40)`

Aquí la situación es analoga al caso anterior salvo que el método recibe dos parametros. El primer parámetro es **&mut self** y al igual que antes, el metodo podrá acceder a las propiedades del objeto pero además de poder leerlas, la palabra **mut** en **&mut self** le permite también modificarlas.

El segundo argumento **daño** opera igual que en una función normal. Se utiliza en el interior del método.

ⓘ Nota: Cuando se llaman los métodos de un struct con el operador punto, el primer parámetro ya sea **&self**, **&mut self** o **self** no es necesario colcarlo.

La función `main()`

En la función `main` se realiza toda la acción.

Se crea el objeto **mi_caza** de la estructura **NaveEspacial** y se **ejecutan sus métodos**.

```
fn main() {
    // Es obligatorio usar 'mut' en mi_caza para poder llamar
    // a métodos que usen &mut self y modifiquen a mi_caza
    // Instanciamos el objeto
    let mut mi_caza = NaveEspacial {
        nombre: String::from("Halcón Milenario"),
        escudo: 100,
        municion: 10,
    };

    mi_caza.reportar_estado(); // Llama al método de lectura
    mi_caza.recibir_disparo(40); // Llama al método de modificación
    mi_caza.reportar_estado(); // Volvemos a leer para comprobar los cambios
}
```

3.3. Funciones asociadas: Constructores personalizados (`new`)

A veces queremos meter una función dentro de un bloque `impl` que **no reciba `self`**. A esto se le llama **función asociada**. Como no tiene `self`, no actúa sobre un objeto ya creado, sino que suele usarse para **fabricar uno nuevo**. En Rust, la convención **para el constructor de un objeto es llamarlo `new`**.

```
impl NaveEspacial {
    // Constructor: No recibe self, porque su trabajo es CREAR el
    // objeto desde cero
    fn new(nombre_nave: &str) -> NaveEspacial {
        NaveEspacial {
            nombre: String::from(nombre_nave),
            escudo: 100, // Todas las naves empiezan con escudo a tope
            municion: 50, // Y munición cargada por defecto
        }
    }
}

fn main() {
    // Para llamar a una función asociada usamos los cuatro puntos (::)
    let nueva_nave = NaveEspacial::new("X-Wing");

    // Si queremos que el programa funcione hay que copiar el método
    // reportar_estado definido anteriormente y pegarlo fuera del main(),
    // por ejemplo arriba del main()
    nueva_nave.reportar_estado();
}
```

El constructor **`new`** que hemos definido crea un objeto de tipo **`NaveEspacial`** con los valores indicados para sus propiedades. Siempre que se invoque este constructor creará el mismo objeto que vemos abajo.

```
NaveEspacial {
    // nombre_nave se recibe en el parámetro de la función
    nombre: String::from(nombre_nave),
    escudo: 100, // Todas las naves empiezan con escudo a tope
    municion: 50, // Y munición cargada por defecto
}
```

Este objeto es lo que devuelve la función **`new`**, ya que está en su última línea y no lleva punto y coma al final.

3.4. 🎮 Proyecto Práctico I: El simulador de inventario de una tienda de videojuegos

Para cerrar este bloque, vamos a unificar las **`structs`**, los vectores (**`vec`**) que aprendimos en el Cuaderno 2, y los bloques **`impl`** con constructores y métodos. Crearemos el motor de gestión para una tienda de videojuegos.

📄 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **`tienda_videojuegos.rs`**

```
// Definimos la estructura de un Videojuego individual
struct Videojuego {
    titulo: String,
    precio: f64,
    stock: u32,
}

impl Videojuego {
    // Constructor de un juego
    fn new(titulo: &str, precio: f64, stock: u32) -> Videojuego {
        Videojuego {
            titulo: String::from(titulo),
            precio,
            stock,
        }
    }
}

// Definimos la estructura de la Tienda, que albergará una
// lista de videojuegos
struct Tienda {
    nombre: String,
    inventario: Vec<Videojuego>, // Usamos un vector dinámico de estructuras
}

impl Tienda {
    // Constructor de la tienda
    fn new(nombre: &str) -> Tienda {
        Tienda {
            nombre: String::from(nombre),
            inventario: Vec::new(), // Empezamos con el inventario vacío
        }
    }

    // Método para añadir un videojuego al inventario
    fn agregar_juego(&mut self, juego: Videojuego) {
        println!("📦 Añadiendo al almacén: {}", juego.titulo);
        self.inventario.push(juego);
    }

    // Método de lectura para listar todos los productos en stock
    fn mostrar_inventario(&self) {
        println!("\n--- 🛒 INVENTARIO DE: {} ---", self.nombre.to_uppercase());
        for juego in &self.inventario {
            println!("• {} | Precio: {:.2}€ | Unidades: {}", juego.titulo,
juego.precio, juego.stock);
        }
        println!("-----\n");
    }

    // Método mutable para simular una venta
    fn vender_juego(&mut self, titulo_juego: &str) {
        let mut encontrado = false;

        for juego in &mut self.inventario {
            if juego.titulo == titulo_juego {
```

```

        encontrado = true;
        if juego.stock > 0 {
            juego.stock -= 1;
            println!("✅ ¡Venta realizada con éxito! Disfruta de: {}",
juego.titulo);
        } else {
            println!("❌ Lo sentimos, no queda stock de: {}",
juego.titulo);
        }
        break; // Salimos del bucle al encontrar el juego
    }
}

if !encontrado {
    println!("🔍 El juego '{}' no se encuentra en nuestro catálogo.",
titulo_juego);
}
}
}

fn main() {
    // 1. Inauguramos nuestra tienda
    let mut mi_tienda = Tienda::new("Pixel & Bits");

    // 2. Creamos y añadimos stock de productos
    let juego1 = Videojuego::new("Rust: Survival Evolved", 39.99, 3);
    let juego2 = Videojuego::new("Cyberpunk 2077", 59.99, 1);

    mi_tienda.agregar_juego(juego1);
    mi_tienda.agregar_juego(juego2);

    // 3. Mostramos el estado inicial
    mi_tienda.mostrar_inventario();

    // 4. Simulamos compras por parte de los clientes
    mi_tienda.vender_juego("Cyberpunk 2077"); // Quedará con stock 0
    mi_tienda.vender_juego("Cyberpunk 2077"); // Debería dar mensaje de falta
de stock
    mi_tienda.vender_juego("Minecraft");      // No existe en la tienda

    // 5. Comprobamos cómo ha quedado el inventario final
    mi_tienda.mostrar_inventario();
}

```

La salida del programa

📦 Añadiendo al almacén: Rust: Survival Evolved
 📦 Añadiendo al almacén: Cyberpunk 2077

```

--- 🛒 INVENTARIO DE: PIXEL & BITS ---
• Rust: Survival Evolved | Precio: 39.99€ | Unidades: 3
• Cyberpunk 2077 | Precio: 59.99€ | Unidades: 1
-----

```

✓ ¡Venta realizada con éxito! Disfruta de: Cyberpunk 2077
 ✗ Lo sentimos, no queda stock de: Cyberpunk 2077
 🔍 El juego 'Minecraft' no se encuentra en nuestro catálogo.

--- 🛒 INVENTARIO DE: PIXEL & BITS ---
 • Rust: Survival Evolved | Precio: 39.99€ | Unidades: 3
 • Cyberpunk 2077 | Precio: 59.99€ | Unidades: 0

Explicación del programa

Vamos a escribir una especie de resumen del programa.

1) Tenemos dos estructuras en nuestro programa: **Videojuego** y **Tienda**.

- La estructura Videojuego tiene solo un constructor **new** en su bloque impl:
 - fn new(titulo: &str, precio: f64, stock: u32) -> Videojuego
- La estructura Tienda tiene un constructor new y tres métodos:
 - fn new(nombre: &str) -> Tienda
 - fn agregar_juego(&mut self, juego: Videojuego)
 - fn mostrar_inventario(&self) {
 - fn vender_juego(&mut self, titulo_juego: &str)

Veamos el constructor new de la estructura Videojuego. Su código es el siguiente:

```
fn new(titulo: &str, precio: f64, stock: u32) -> Videojuego {
    Videojuego {
        titulo: String::from(titulo),
        precio,
        stock,
    }
}
```

Teóricamente lo tendríamos que haber escrito así:

```
fn new(titulo: &str, precio: f64, stock: u32) -> Videojuego {
    Videojuego {
        titulo: String::from(titulo),
        precio: precio,
        stock: stock,
    }
}
```

Pero como los parámetros **precio** y **stock** se llaman igual que las propiedades correspondientes del Videojuego, es suficiente con escribirlo una sola vez como hacemos arriba.

Utilización de los métodos en el main.

Creación de un videojuego.

- El constructor a utilizar:
 - fn new(titulo: &str, precio: f64, stock: u32) -> Videojuego

- La instrucción en el main:
 - `let juego1 = Videojuego::new(«Rust: Survival Evolved», 39.99, 3);`

Creación de la tienda:

- El constructor a utilizar:
 - `fn new(nombre: &str) -> Tienda`
- La instrucción en el main:
 - `let mut mi_tienda = Tienda::new("Pixel & Bits");`

Agregar un juego a la tienda:

- El método a utilizar:
 - `fn agregar_juego(&mut self, juego: Videojuego)`
- La instrucción en el main:
 - `mi_tienda.agregar_juego(juego1);`

Mostrar inventario de la tienda

- El método a utilizar:
 - `fn mostrar_inventario(&self)`
- La instrucción en el main:
 - `mi_tienda.mostrar_inventario();`

Vender un juego

- El método a utilizar:
 - `fn vender_juego(&mut self, titulo_juego: &str)`
- La instrucción en el main:
 - `mi_tienda.vender_juego("Cyberpunk 2077");`

Te queda un importante trabajo inspeccionando el interior de los métodos y observando:

- Como devuelven los constructores **new** los respectivos objetos.
- Como se utiliza el **operador punto** para acceder a las propiedades de un objeto, por ejemplo **juego.titulo**.
- Diferenciar entre métodos que solo leen las propiedades del objeto y reciben como argumento **&self** de los que también pueden modificar el objeto recibiendo **&mut self**.

4. Enumerados (enum): La Joya de la Corona de Rust

Hasta ahora hemos aprendido a usar estructuras (`struct`) para modelar objetos: una nave espacial siempre tiene por lo menos un **nombre** (la identidad de la nave), un **escudo** (la capacidad de defensa o los puntos de vida) y **munición** (la cantidad de disparos disponibles). Pero, ¿qué pasa si queremos representar un concepto que puede manifestarse de diferentes formas **mutuamente excluyentes**?

Por ejemplo, la poción de un videojuego puede ser de *Salud*, *Magia* o *Fuerza*. Un usuario puede iniciar sesión mediante *Email*, *Teléfono* o *Google*. Para resolver esto con total seguridad en los tipos, Rust utiliza los enumerados o enumeraciones (`enum`).

Una enumeración es una variable que puede adoptar un valor entre un conjunto finito de variantes. Además, en Rust, cada variante puede tener datos asociados como veremos a continuación.

4.1. Enumeraciones con datos asociados

En otros lenguajes de programación, un enumerado o enumeración es simplemente una lista de etiquetas o variantes. En Rust, los enumerados son un «super-tipo» porque cada variante puede **almacenar sus propios datos con estructuras diferentes**.

Imagina el sistema de hechizos de un videojuego RPG:

📌 Nota: Un juego RPG (Role-Playing Game) significa que es un juego de rol, donde controlas a un personaje o a un grupo mientras mejoran sus habilidades, suben de nivel y avanzan por una historia profunda

Abajo definimos la enumeración **Hechizo** con sus tres variantes: la variante **Curación** no tiene datos asociados, la variante **BolaFuego** puede contener un `struct` y la variable **Teletransporte** puede almacenar una tupla.

```
// Un hechizo SOLO puede ser de una de las tres variantes
// que se definen en el enum Hechizo:
// Curacion: Variante tradicional (sin datos)
// BolaFuego: Variante con una estructura interna
// Teletransporte: Variante con una tupla interna (Coordenadas X, Y)
enum Hechizo {
    Curacion,
    BolaFuego { daño: u32, radio: f32 },
    Teletransporte(i32, i32),
}
```

Para instanciar una variante de un enumerado, utilizamos el nombre del enumerado seguido de los dos puntos dobles (`::`):

```
enum Hechizo {
    Curacion,
    BolaFuego { daño: u32, radio: f32 },
    Teletransporte(i32, i32),
}

fn main() {
    // Creamos tres variables que son del mismo tipo "Hechizo"
    // pero cada una representa una variante de Hechizo
    let hechizo_basico = Hechizo::Curacion;

    let hechizo_ataque = Hechizo::BolaFuego {
        daño: 50,
        radio: 4.5,
    };

    let escape_emergencia = Hechizo::Teletransporte(150, -80);
}
```

Nota: Las tres variables comparten el tipo Hechizo, pero cada una guarda la información exacta que necesita para funcionar.

4.2. El tipo Option<T>: La solución definitiva al peligro del valor «Nulo»

¿Que debe devolver una función que busca en una base de datos el registro de una persona a partir de su DNI, si el DNI que se le pasa a la función para que busque no existe en la Base de datos?. Es algo que hay que tener en cuenta en el diseño del lenguaje de programación.

En la gran mayoría de lenguajes tradicionales (como Java, C++ o JavaScript) existe el concepto de valor **Nulo** (null o nil). Es el contenido de una variable que apunta al vacío (un lugar de la memoria en donde no existe información útil para el programa). Cuando intentas usar un valor nulo, provoca el error informático más famoso y temido del mundo: el **NullPointerException**, que hace que los programas se congelen y crasheen.

En Rust no existe el valor Nulo. ¡El creador del lenguaje lo eliminó por completo! En su lugar, Rust utiliza un enumerado nativo ultra inteligente llamado Option<T>.

Este enumerado viene integrado (incluido) en el lenguaje y está definido de la siguiente manera:

```
// T puede ser cualquier tipo
enum Option<T> {
    Some(T),          // Significa: "Aquí hay un valor válido de tipo T"
    None,             // Significa: "No hay ningún valor, está vacío"
}
```

Decir que **Option<T>** viene integrado en el lenguaje equivale a que podemos utilizar sus dos variantes sin definir en nuestro código la enumeración Option.

La **T** que aparece en Option<T> significa «dato genérico» es decir, cualquier tipo de dato. Nosotros manejaremos por ejemplo **Option<string>** con sus dos variantes **Some(string)** y

None. ¿Cómo interpretamos esto? Si una función devuelve un dato de tipo **Option<String>** entonces tiene que devolver o bien **Some(String)** o **None**. Tendremos que averiguar cual es la variante devuelta y en el caso de que sea **Some<String>** averiguar cuanto vale el **String** escondido en el **Some**.

Como veremos, cuando una variable recibe un dato de tipo **Option<T>**, que solo puede ser **Some(T)** o **None**, obliga al programador a gestionar explícitamente el caso en el que un dato no exista, haciendo imposible que el programa falle por sorpresa.

Existen multitud de recursos como **match**, **if let**, etc. para averiguar qué variante contiene la variable y si es **Some**, averiguar también el dato que esconde que suele ser el resultado.

Ejemplo con Option<String>: El inventario de armas

En este ejemplo definimos el struct **Personaje** con los campos **nombre** (un **String**) y **arma equipada** del tipo **Option<String>**. **arma equipada** la definimos de tipo **Option<String>** porque podría valer o bien **Some("nombre del arma")** o **None** en caso de que no lleve arma. En este ejemplo utilizaremos funciones que nos dicen si **arma equipada** es la variante **Some** o la **None**, pero no averiguaremos el nombre del arma.

Imagina un personaje que puede o no equipar un arma en su mano derecha.

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **option_arma.rs**

```
struct Personaje {
    nombre: String,
    // El arma es opcional: puede tener un String con el nombre del arma,
    // o no tener nada
    arma equipada: Option<String>,
}

fn main() {
    // Caso 1: Un guerrero armado
    let guerrero = Personaje {
        nombre: String::from("Conan"),
        arma equipada: Option::Some(String::from("Espada Atlante")),
    };

    // Caso 2: Un monje que pelea con los puños libres
    let monje = Personaje {
        nombre: String::from("Shaolin"),
        arma equipada: Option::None, // Expresamos la ausencia de datos con
total seguridad
    };

    // Rust nos da métodos rápidos para comprobar el contenido de un Option:
    if guerrero.arma equipada.is_some() {
        println!("🗡 {} está listo para el combate.", guerrero.nombre);
    }

    if monje.arma equipada.is_none() {
        println!("🙏 {} prefiere la diplomacia y sus puños.", monje.nombre);
    }
}
```

```
}
}
```

¿Cómo extraemos de forma segura el texto "Espada Atlante" que está atrapado dentro de Some?

4.3. La instrucción match

En el siguiente apartado nos detendremos para ver de forma detallada como utilizar la instrucción **match** para **desempaquetar** un **Option**, es decir, para averiguar lo que contiene, si es **Some** o es **None**. Además, en el caso de que sea **Some** averiguaremos que dato tiene dentro.

Sin embargo, en Rust, **match** es también una herramienta de control de flujo ultrapotente que funciona como un "if-else" tradicional con esteroides, y que además tiene la capacidad de «abrir» estructuras de datos como hemos comentado antes.

Aquí tienes una explicación resumida dividida en sus dos facetas principales:

- **Control de flujo general:** Funciona como un if/else extendido o un switch avanzado de otros lenguajes. Compara un valor con múltiples opciones (patrones) de forma exhaustiva y ejecuta el bloque de código de la primera opción que coincida.
- **Desempaquetado (Pattern Matching):** Es el mecanismo nativo para extraer de forma segura el valor dentro de tipos complejos como **Option** (sacar el valor si es **Some** o manejar el caso si es **None**), obligándote por diseño a gestionar todos los escenarios posibles y evitando errores en tiempo de ejecución.

Ejemplo de código

Nos anticipamos un poco al siguiente tema con el objetivo de mostrar la capacidad de la instrucción **match** sin vincularla únicamente al desempaquetado de datos de tipo **Option**.

El siguiente listado presenta los dos casos de utilización de **match** que hemos comentado.

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **option_ejemplo.rs**

```
fn main() {
    // 1. Como control de flujo general
    let numero = 2;
    match numero {
        1 => println!("Es uno"),
        2 | 3 => println!("Es dos o tres"),
        _ => println!("Es cualquier otro número"), // Es como el "else"
    }

    // 2. Para desempaquetar un Option
    let opcional = Some(1);
    match opcional {
        Some(dato) => println!("El valor interno es: {}", dato),
        None => println!("No hay ningún valor"),
    }
}
```

```
}
}
```

El caso 1. es un caso sencillo de control de flujo con match.

- Inicializamos la variable **numero** con un 2.
- Luego el match pregunta. **match número** es lo mismo que decir **si numero vale...**
 - si vale 1 => println!("Es uno")
 - si vale 2 o 3 => println!("Es dos o tres")
 - si vale _ (cualquier otra cosa) => println!("Es cualquier otro número")
- Resultado: **"Es dos o tres"**

En el caso 2. el match desempaqueta la variable opcional. Le estamos diciendo al match. Si la variante de opcional es Some, extrae su valor en la variable dato e imprime: println!(«El valor interno es: {}», dato). Si la variante es None, imprime: println!(«No hay ningún valor»)

- Resultado: "El valor interno es: 1"

📌 Nota: Hemos podido utilizar la expresión Some(1) sin definir nada antes porque el enumerado Option<T> con sus dos variantes Some(T) y None están definidas ya en el sistema por Rust

En el siguiente tema seguimos hablando de match.

5. Control de Flujo Avanzado: Coincidencia de Patrones (`match`)

En el tema anterior descubrimos las enumeraciones (`enum`) y cómo pueden almacenar datos en sus variantes. Pero nos quedamos con una duda: ¿cómo podemos «abrir» esas variantes de forma segura para extraer y utilizar los datos que contienen?

Nota: En el capítulo anterior vimos un ejemplo sencillo de una variable de tipo `Option` que almacenaba `Some(1)` y pudimos extraer ese `1` mediante un `match`.

Para conseguirlo, Rust nos ofrece varias posibilidades pero la estructura de control definitiva es **`match` (coincidencia de patrones)**. Piensa en `match` como un observador de tu variable **`enum`**. A través de `match` el compilador inspecciona tu **variable `enum`** y te obliga a programar un camino específico para cada una de las variantes posibles.

5.1. La potencia de `match`: Desestructuración exhaustiva de datos

Una de las reglas de oro de Rust es la **exhaustividad**: un bloque `match` no compilará si dejas una sola variante de la enumeración sin cubrir.

Recuperemos el ejemplo de los hechizos del apartado anterior para ver cómo `match` abre el contenido y extrae las variables internas (proceso llamado desestructuración):

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: `match_hechizo.rs`

```
enum Hechizo {
    Curacion,
    BolaFuego { daño: u32, radio: f32 },
    Teletransporte(i32, i32),
}

fn lanzar_hechizo(hechizo: Hechizo) {
    // El match inspecciona la variante exacta y extrae sus valores internos
    match hechizo {
        Hechizo::Curacion => {
            println!("♥ Destellos verdes flotan en el aire. Te has curado 20 puntos de vida.");
        }
        Hechizo::BolaFuego { daño, radio } => {
            println!("🔥 ¡BOOM! Una bola de fuego estalla haciendo {} de daño en un radio de {} metros.", daño, radio);
        }
        Hechizo::Teletransporte(x, y) => {
            println!("🌀 Te desvaneces en el espacio y apareces en las coordenadas (X: {}, Y: {}).", x, y);
        }
    }
}
```

```

}

fn main(){
    let hechizo = Hechizo::BolaFuego{daño: 8, radio: 20.7f32};
    lanzar_hechizo(hechizo);
}

```

La ejecución del programa produce la siguiente salida:

🔥 ¡BOOM! Una bola de fuego estalla haciendo 8 de daño en un radio de 20.7 metros.

Explicación

Fijémonos en primer lugar en la función **lanzar_hechizo(hechizo: Hechizo)**. Recibe como argumento una variable de tipo **Hechizo**.

A continuación y dentro de la función, mediante **match hechizo** averigua qué **variante** es la variable **hechizo** y para cada una de las tres posibilidades establece el código que se ejecutará.

En el **main()** creamos la variable **hechizo = Hechizo::BolaFuego{daño: 8, radio: 20.7f32};** que le pasamos a la función.

Es el segundo caso dentro de la función. El **match** hace que se ejecute el código entre llaves que reproducimos a continuación:

```

Hechizo::BolaFuego { daño, radio } => {
    println!("🔥 ¡BOOM! Una bola de fuego estalla haciendo {} de daño en un
radio de {} metros.", daño, radio);
}

```

Observa que podríamos haber escrito el código de esta otra forma:

```

Hechizo::BolaFuego { variable1, variable2 } => {
    println!("🔥 ¡BOOM! Una bola de fuego estalla haciendo {} de daño en un
radio de {} metros.", variable1, variable2);
}

```

Miremos de nuevo en el **main()** el valor de **hechizo** que le pasamos a la función:

```

let hechizo = Hechizo::BolaFuego{daño: 8, radio: 20.7f32};

```

Por tanto, la **variable1** tomará el valor 8 y la **variable2** 20.7f32. Esto es independiente del nombre que le demos a esas variables, solo influye el orden al escribirlas.

5.2. El comodín de seguridad: El patrón por defecto o guión bajo _

En este apartado consideramos a **match** como una instrucción de **control de flujo**, similar a **if-else** pero con más poder. A veces trabajamos con tipos de datos que tienen millones de posibilidades, como un número entero (**u32**) o un carácter (**char**). Sería imposible en un **match**, escribir un camino para cada número. Para solucionarlo, usamos el guión bajo (**_**), que

actúa como un «comodín» o caso por defecto para atrapar todo lo que no hayamos atrapado explícitamente.

La siguiente función **clasificar_puntuacion** tiene un parámetro **puntos** de tipo **u32** (entero sin signo de 32 bits) y en función del argumento que se le pase a la función, ejecuta una u otra rama del **match**. Vemos que al principio hay tres caminos perfectamente definidos pero el cuarto camino se recoge con el patrón guión bajo **_** que atrapa cualquier valor que no se haya atrapado antes.

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **clasificar_puntuacion.rs**

```
fn clasificar_puntuacion(puntos: u32) {
    match puntos {
        0 => println!("🏆 ¿Es tu primera vez jugando? Ánimo."),
        1..=10 => println!("🥈 ¡Vas mejorando! Medalla de plata."),
        11..=50 => println!("🥇 ¡Increíble! Eres un profesional."),
        _ => println!("🏆 ¡Récord legendario superado!"), // Captura cualquier
número mayor que 50
    }
}

fn main(){
    clasificar_puntuacion(0);
    clasificar_puntuacion(5);
    clasificar_puntuacion(25);
    clasificar_puntuacion(100);
}
```

La ejecución del programa produce la siguiente salida:

```
🏆 ¿Es tu primera vez jugando? Ánimo.
🥈 ¡Vas mejorando! Medalla de plata.
🥇 ¡Increíble! Eres un profesional.
🏆 ¡Récord legendario superado!
```

Como ya dijimos al principio aquí no estamos utilizando **match** para desempaquetar una enumeración de tipo **Option** sino simplemente para decidir que rama de código se va a ejecutar dependiendo del argumento pasado a una función.

5.3. El atajo elegante: Control de flujo simplificado con **if let**

¿Qué ocurre si solo nos interesa gestionar **una sola variante** de una **enumeracion** y queremos ignorar el resto? Usar un **match** nos obligaría a poner el comodín **_ => {}** al final de forma obligatoria. Para evitar ese código repetitivo, Rust inventó el atajo **if let**.

if let es útil cuando sabemos que una variable de tipo **Option** es la variante **Some** y sólo queremos averiguar el dato que encierra el **Some**. Entonces podemos hacerlo sin riesgo con una sola línea de código.

En el siguiente ejemplo, la variable **cofre_tesoro** está inicializada con la variante **Some** de un `Option<String>`. En el programa suponemos que solo nos interesa saber lo que contiene la variable **cofre_tesoro** si es un **Some** y nos da igual en caso contrario.

El **if let**, si la variable **cofre_tesoro** coincide exactamente con el patrón **Some**, extrae el texto en el argumento **arma** que le pasamos al **Some**. Es decir, **arma** pasará a valer "Espada Excalibur" y la podremos utilizar en el `println!`.

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **cofre_tesoro.rs**

```
fn main() {
    let cofre_tesoro: Option<String> = Option::Some(String::from("Espada
    Excalibur"));

    if let Option::Some(arma) = cofre_tesoro {
        println!("🎁 ¡Has abierto un cofre y has encontrado una {}!", arma);
    }
}
```

La salida del programa es:

🎁 ¡Has abierto un cofre y has encontrado una Espada Excalibur!

`if let` serviría igual para desempaquetar la variante **None** pero en este caso, si sabemos ya seguro que es **None**, no tenemos nada más que averiguar y el `if let` se hace innecesario.

5.4. Proyecto Práctico II: Mini-RPG de texto basado en turnos

Vamos a unir todo lo aprendido en el Cuaderno 3 (estructuras, métodos `impl`, enumeraciones `Option` y control de flujo `match`) para crear un simulador de combate clásico por turnos entre un Héroe y un Monstruo.

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **combatiente.rs**

Después del listado damos algunas explicaciones.

```
// 1. Definimos las acciones que puede tomar el jugador en su turno
enum Accion {
    Atacar,
    Curar,
    Huir,
}

// 2. Definimos la estructura de los combatientes
struct Combatiente {
    nombre: String,
    salud: i32,
    salud_maxima: i32,
    fuerza_ataque: i32,
}
```

```

impl Combatiente {
    // Constructor de personajes
    fn new(nombre: &str, salud_max: i32, fuerza: i32) -> Combatiente {
        Combatiente {
            nombre: String::from(nombre),
            salud: salud_max,
            salud_maxima: salud_max,
            fuerza_ataque: fuerza,
        }
    }

    // Método para recibir un impacto
    fn recibir_daño(&mut self, cantidad: i32) {
        self.salud -= cantidad;
        if self.salud < 0 {
            self.salud = 0;
        }
        println!("💥 {} recibe {} puntos de daño. (Vida actual: {}/{})",
            self.nombre, cantidad, self.salud, self.salud_maxima);
    }

    // Método para curarse
    fn usar_pocion(&mut self) {
        let curacion = 25;
        self.salud += curacion;
        if self.salud > self.salud_maxima {
            self.salud = self.salud_maxima;
        }
        println!("❤️ {} bebe una poción y recupera {} de vida. (Vida actual: {}/{})",
            self.nombre, curacion, self.salud, self.salud_maxima);
    }

    // Comprobar si el personaje sigue en pie
    fn esta_vivo(&self) -> bool {
        self.salud > 0
    }
}

fn main() {
    println!("=== 🏰 BIENVENIDO A RUST-RPG 🏰 ===");

    // Instanciamos a los combatientes
    let mut heroe = Combatiente::new("Sir Isaac", 80, 18);
    let mut monstruo = Combatiente::new("Gólem de Roca", 100, 12);

    // Simulamos la lista de comandos/turnos predefinidos del jugador
    let turnos_jugador = [Accion::Atacar, Accion::Curar, Accion::Atacar,
        Accion::Huir];
    let mut numero_turno = 1;

    for accion in turnos_jugador {
        if !heroe.esta_vivo() || !monstruo.esta_vivo() {
            break;
        }
    }
}

```

```
println!("\n--- 🕒 TURNO {} ---", numero_turno);

// --- FASE DEL HÉROE ---
// Usamos match para decidir qué superpoder o acción ejecuta el jugador
match accion {
    Accion::Atacar => {
        println!("🗡️ {} alza su espada contra el {}!", heroe.nombre,
monstruo.nombre);
        monstruo.recibir_daño(heroe.fuerza_ataque);
    }
    Accion::Curar => {
        heroe.usar_pocion();
    }
    Accion::Huir => {
        println!("🏃 {} ha decidido retirarse del combate de forma
segura. ¡Fin de la partida!", heroe.nombre);
        return; // Corta la ejecución completa del main
    }
}

// --- FASE DEL ENEMIGO (Si sigue vivo) ---
if monstruo.esta_vivo() {
    println!("😡 El {} ruge con furia y contraataca!",
monstruo.nombre);
    heroe.recibir_daño(monstruo.fuerza_ataque);
}

numero_turno += 1;
}

// --- RESOLUCIÓN DEL COMBATE ---
println!("\n=====");
if heroe.esta_vivo() && !monstruo.esta_vivo() {
    println!("🏆 ¡VICTORIA! El {} ha sido derrotado. Eres el héroe del
reino.", monstruo.nombre);
} else if !heroe.esta_vivo() && monstruo.esta_vivo() {
    println!("💀 HAS MUERTO. El {} ha protegido su mazmorra con éxito.",
monstruo.nombre);
} else {
    println!("🔪 El combate ha quedado en tablas.");
}
println!("\n=====");
}
```

Explicaciones sobre el listado

La mejor forma de seguir este apartado es editando en la Playground el listado del programa e ir identificando las explicaciones con el listado. Ten en cuenta que no se van a explicar las líneas una por una en todos los casos sino que a menudo explicaremos grupos de líneas.

1. Lo primero es indicar las estructuras de datos que tenemos:

- **enum Accion:** Tiene tres variantes, **Atacar**, **Curar** y **Huir**

- **struct Combatiente:** Tiene cuatro campos. Utilizaremos este struct para crear tanto el **heroe** como el **monstruo**.
- **impl Combatiente:** Define el constructor **new()**, y los métodos **recibir_daño()**, **usar_pocion()** y **esta_vivo()** que pueden actuar sobre un objeto de tipo **Combatiente**. El constructor **new()** simplemente lo crea.

2. La función main

- En primer lugar crea el **heroe** y el **monstruo** con el constructor **new** del **struct Combatiente**. Utiliza **mut** para poder luego cambiar sus propiedades.

```
let mut heroe = Combatiente::new("Sir Isaac", 80, 18);
let mut monstruo = Combatiente::new("Gólem de Roca", 100, 12);
```

- A continuación crea dos variable más:

```
let turnos_jugador = [Accion::Atacar, Accion::Curar, Accion::Atacar,
Accion::Huir];
let mut numero_turno = 1;
```

turnos_jugador es un array con 4 acciones que luego recorreremos con un bucle for.

La variable **numero_turno** se inicializa a 1 antes de entrar en el for y luego se aumenta en uno al final del for en cada pasada del mismo. Solo se utiliza para imprimir el número de turno del for e irá tomando los valores 1, 2, 3 y 4.

3. El bucle for Se introduce con la siguiente línea:

```
for accion in turnos_jugador {
    // instrucciones del for
}
```

Hemos visto antes que **turnos_jugador** es un array de cuatro acciones. El bucle for recorrerá las cuatro posiciones del array entregando a la variable **accion** el valor correspondiente en cada pasada. Por ejemplo, en la primera pasada, **accion** valdrá **Accion::Atacar**, en la segunda valdrá **Accion::Curar** y así sucesivamente. En cada pasada se ejecutan todas las instrucciones del bucle.

4. El match Cuando el bucle for llega al **match**, desempaqueta la variable **accion** y según su contenido ejecuta una de sus ramas:

```
match accion {
    Accion::Atacar => {
        println!("⚔ {} alza su espada contra el {}!", heroe.nombre,
monstruo.nombre);
        monstruo.recibir_daño(heroe.fuerza_ataque);
    }
    Accion::Curar => {
```

```

        hero.e.usar_pocion();
    }
    Accion::Huir => {
        println!("🏃 {} ha decidido retirarse del combate de forma segura. ¡Fin de la partida!", hero.nombre);
        return; // Corta la ejecución completa del main
    }
}

```

5. ¿El monstruo sigue vivo? (Al final del for)

Si sigue vivo se le aplica:

```

hero.e.recibir_daño(monstruo.fuerza_ataque);

```

¿Cuanto vale **monstruo.fuerza_ataque**? Según hemos creado el monstruo con la instrucción `new`, `fuerza_ataque` es el tercer argumento, luego vale 12.

6. Evolución del for

El bucle `for` irá avanzando entre las cuatro acciones del array **turnos_jugador** y para cada una de ellas ejecutará todas las instrucciones existentes entre su llave de apertura y su llave de cierre.

7. Impresión de resultados

Después de que el `for` de sus cuatro vueltas, las últimas líneas imprimen el resultado del combate según como hayan quedado los estados del `hero` y del `monstruo`.

8. Los resultados

Estos son los resultados que debes obtener ejecutando el programa:

🏰 BIENVENIDO A RUST-RPG 🏰

— ⌚ TURNO 1 —

⚔️ Sir Isaac alza su espada contra el Gólem de Roca!

💥 Gólem de Roca recibe 18 puntos de daño. (Vida actual: 82/100)

🐼 El Gólem de Roca ruga con furia y contraataca!

💥 Sir Isaac recibe 12 puntos de daño. (Vida actual: 68/80)

— ⌚ TURNO 2 —

💚 Sir Isaac bebe una poción y recupera 25 de vida. (Vida actual: 80/80)

🐼 El Gólem de Roca ruga con furia y contraataca!

💥 Sir Isaac recibe 12 puntos de daño. (Vida actual: 68/80)

— ⌚ TURNO 3 —

⚔️ Sir Isaac alza su espada contra el Gólem de Roca!

💥 Gólem de Roca recibe 18 puntos de daño. (Vida actual: 64/100)

 El Gólem de Roca ruga con furia y contraataca!

 Sir Isaac recibe 12 puntos de daño. (Vida actual: 56/80)

—  TURNO 4 —

 Sir Isaac ha decidido retirarse del combate de forma segura. ¡Fin de la partida!

6. El mapa definitivo de la seguridad: Option y Result

Hasta ahora hemos ido utilizando la instrucción **match** básicamente en dos escenarios: como instrucción de control de flujo y para desempaquetar una variable de tipo **Option**. Aunque este segundo caso está ligado a la seguridad del programa, el control eficiente de errores se consigue introduciendo también la enumeración **Result** y un conjunto de métodos como **unwrap()**, **unwrap_or()**, **expect()**, etc. como herramientas que de forma ocasional actúan como pequeños «salvavidas» en nuestro código para evitar que el programa se quede colgado (se congele). Pero va siendo hora de levantar el capó para entender en qué consiste realmente el tratamiento de errores de forma estructurada y por qué hacen a Rust un lenguaje tan único.

En Rust no existen los valores **null** o **nil** que en otros lenguajes causan tantos dolores de cabeza y cuelgues inesperados. En su lugar, el lenguaje utiliza dos **enumeraciones** llamadas **Option** y **Result**. Como son tan fundamentales para la supervivencia de cualquier programa, Rust las incluye directamente en su prelude (su caja de herramientas básica). Esto significa que ya están definidas en el corazón del lenguaje y las podemos utilizar sin importarlas ni configurar nada.

Para entender cómo funcionan, tenemos que mirar su definición utilizando Genéricos, que en Rust representamos con letras mayúsculas como **<T>**, **<E>** o **<V>**. No dejes que la sintaxis te asuste: estas letras son simplemente «cajas vacías» o comodines que se sustituirán por tipos de datos reales cuando escribamos nuestro código.

6.1. La caja de la presencia o ausencia: Option<T>

Recordemos que el tipo **Option** se utiliza cuando algo puede existir o puede no existir. Por ejemplo, una función que busque un usuario en una base de datos a través de su DNI puede devolver un dato de tipo **Option**. En este caso devolvería el usuario encontrado envuelto en la variante **Some** o directamente **None** si no encontró el usuario. Esto puede ocurrir porque proporcionamos un DNI que no existe en la base de datos o porque en ese momento no se pudo realizar la conexión con la base de datos.

Como ya vimos, el lenguaje lo define así:

```
enum Option<T> {
    Some(T), // Contiene un valor válido de tipo T
    None,    // No contiene nada
}
```

Esa **<T>** es el tipo genérico del valor que viaja dentro de la variante **Some** si las cosas van bien. En la práctica, cuando trabajamos con geometría o física, solemos usar **Option<f64>**, lo que significa que, si hay éxito, la variante **Some** envolverá un número decimal de 64 bits (**f64**), y si no, nos devolverá un rotundo **None**.

6.2. La caja del éxito o el fracaso: Result<T, E>

LLegó el momento de analizar la enumeración **Result**.

El tipo **Result** va un paso más allá. No se pregunta si algo existe o no, sino **si una operación ha salido bien o ha fallado**. En caso de éxito nos entrega el resultado en la variante **Ok** y en caso de fallo, nos explica el motivo en la variante **Err**. Su definición genérica utiliza dos comodines:

```
enum Result<T, E> {
    Ok(T),      // La operación fue un éxito y nos da un valor de tipo T
    Err(E),     // Hubo un error y nos da un detalle del fallo de tipo E
}
```

Aquí la magia es doble: **<T>** representa el tipo de dato del resultado feliz, y **<E>** representa el tipo de dato del error.

En tus primeros programas verás a menudo combinaciones sencillas como **Result<f64, string>**. Esto significa que si todo va bien, con **Ok** obtenemos nuestro número f64, y si algo falla, con **Err** recibimos un texto explicativo (String) con los motivos del desastre.

Perdiendo el miedo: Otros tipos de Result

Aunque usar String para los errores es ideal para aprender y resolver problemas sencillos, Rust está lleno de variantes preparadas para el mundo real. Por ejemplo, cuando trabajamos leyendo archivos en el disco duro o gestionando sockets de red, el lenguaje sustituye la **E** genérica por tipos específicos como **std::io::Error**.

Ver una firma como **Result<String, std::io::Error>** no debe darte ningún miedo: funciona exactamente igual que las demás. La única diferencia es que, en caso de fallo, en lugar de proporcionar un texto plano como un String, la variante **Err** nos entregará un objeto con información técnica precisa proporcionada por el propio sistema operativo. Obviamente, tendremos que saber manejar ese objeto para sacarle la información que almacena.

6.3. El Arsenal de Extracción: ¿Cómo destapamos las cajas?

Para desarrollar este apartado trabajaremos con dos funciones:

- la función: **fn volumen_cubo(lado: f64) -> Option<f64>**
 - Esta función tiene como parámetro el lado de un cubo y calcula su volumen. Para poder realizar nuestro ejercicio, hemos supuesto que si el lado que le proporcionamos a la función es menor que cinco, el cubo no existe (no tenemos cubos tan pequeños) y devolverá **None**. En caso contrario devolverá el volumen del cubo envuelto en un **Some**.

```
fn volumen_cubo(lado: f64) -> Option<f64> {
    if lado < 5.0 {
        // no existe
        return None;
    }

    // resultado válido envuelto en Some()
    let volumen = lado * lado * lado;
    // Como es la última línea, aquí NO se usa 'return' ni punto y coma
    Some(volumen)
}
```

- la función: **fn superficie(lado: f64) -> Option<f64>**
 - Esta función tiene también como parámetro el lado del cubo y calcula su superficie. Hemos supuesto también que si el lado que le proporcionamos a la función es menor que cinco, en este caso incurrimos en un error y entonces proporcionamos un mensaje explicativo del error envuelto en un **Err**. En caso contrario devolverá la superficie del cubo envuelto en un **Ok**.

```
fn superficie_cubo(lado: f64) -> Result<f64, String> {
    if lado < 5.0 {
        // El return es obligatorio para salir corriendo de la
        // función AQUÍ mismo
        return Err(format!("El lado ({}), es muy pequeño. Mínimo debe ser 5.0",
lado));
    }

    let superficie = 6.0 * lado * lado;
    // Como es la última línea, aquí NO se usa 'return' ni punto y coma
    Ok(superficie)
}
```

Resumiendo

- La función **volumen_cubo()** devuelve un **Option**. Concretamente devuelve la variante **None** si le pasamos un lado menor que 5.0 y el volumen del cubo envuelto en la variante **Some** en caso contrario.
- La función **superficie_cubo()** devuelve un **Result**. Concretamente devuelve un String envuelto en la variante **Err** si le pasamos un lado menor que 5.0 y la superficie del cubo envuelta en la variante **Ok** en caso contrario.

6.3.1. El cirujano del código: La sentencia match

Es la forma más explícita y robusta. Te obliga a contemplar todos los escenarios posibles (el éxito y el fracaso). Si te dejas uno, el compilador no te dejará avanzar.

Recordemos cómo desempaquetar el **Option** que devuelve la función que calcula el volumen de nuestro cubo. Esto ya lo hemos hecho antes:

```
fn volumen_cubo(lado: f64) -> Option<f64> {
    if lado < 5.0 {
        // no existe
        return None;
    }

    // resultado válido envuelto en Some()
    let volumen = lado * lado * lado;
    // Como es la última línea, aquí NO se usa 'return' ni punto y coma
    // para devolver Some(volumen)
    Some(volumen)
}

fn main(){
    let resultado_opt = volumen_cubo(6.0); // Devuelve Option<f64>
```

```
match resultado_opt {
    Some(vol) => println!("¡Éxito! El volumen extraído es {} m³", vol),
    None => println!("Lo sentimos. No existen cubos tan pequeños"),
}
```

- Si la variante es **Some**, el match extrae dinámicamente el volumen del cubo, lo coloca en la variable **vol** del **Some** y ejecuta el println!.
- Si la variante es **None** simplemente ejecuta el println!

Podríamos decirlo también así: Si la caja es un **Some**, Rust extrae el número decimal, lo bautiza temporalmente como **vol** y ejecuta esa línea (el println!). Si la caja venía vacía (**None**), ejecuta la otra línea println!.

La salida del programa es:

```
¡Éxito! El volumen extraído es 216 m³
```

Veamos ahora cómo de forma totalmente análoga desempaquetamos el **Result** que devuelve la función `superficie_cubo`:

```
fn superficie_cubo(lado: f64) -> Result<f64, String> {
    if lado < 5.0 {
        // El return es obligatorio para salir corriendo de la
        // función AQUÍ mismo
        return Err(format!("El lado ({{}}) es muy pequeño. Mínimo debe ser 5.0",
            lado));
    }

    let superficie = 6.0 * lado * lado;
    // Como es la última línea, aquí NO se usa 'return' ni punto y coma
    // para devolver Some(superficie)
    Ok(superficie)
}

fn main() {
    let resultado_sup = superficie_cubo(4.0); // Devuelve Result<f64, String>
    match resultado_sup {
        Ok(sup) => println!("La superficie calculada es {} m²", sup),
        Err(mensaje_error) => println!("Fallo en el cálculo: {}",
            mensaje_error),
    }
}
```

- Si la variante es **Ok** el match extrae dinámicamente la superficie del cubo, la coloca en la variable **sup** del **Ok** y ejecuta el println!.
- Si la variante es **Err** el match extrae dinámicamente el mensaje de error, lo coloca en la variable **mensaje_error** del **Err** e imprime el otro println!.

La salida del programa es la siguiente:

```
Fallo en el cálculo: El lado (4) es muy pequeño. Mínimo debe ser 5.0
```

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Tienes los dos ejemplos anteriores en un solo programa.

Fichero: **match_cubo.rs**

```
fn volumen_cubo(lado: f64) -> Option<f64> {
    if lado < 5.0 {
        // no existe
        return None;
    }

    // resultado válido envuelto en Some()
    let volumen = lado * lado * lado;
    // Como es la última línea, aquí NO se usa 'return' ni punto y coma
    Some(volumen)
}

fn superficie_cubo(lado: f64) -> Result<f64, String> {
    if lado < 5.0 {
        // El return es obligatorio para salir corriendo de la
        // función AQUÍ mismo
        return Err(format!(
            "El lado ({{}}) es muy pequeño. Mínimo debe ser 5.0",
            lado
        ));
    }

    let superficie = 6.0 * lado * lado;
    // Como es la última línea, aquí NO se usa 'return' ni punto y coma
    Ok(superficie)
}

fn main() {
    let resultado_vol = volumen_cubo(6.0); // Devuelve Option<f64>
    match resultado_vol {
        Some(vol) => println!("¡Éxito! El volumen extraído es {{}} m³", vol),
        None => println!("Error: El lado indicado no era válido para calcular
el volumen."),
    }

    let resultado_sup = superficie_cubo(4.0); // Devuelve Result<f64, String>
    match resultado_sup {
        Ok(sup) => println!("La superficie calculada es {{}} m²", sup),
        Err(mensaje_error) => println!("Fallo en el cálculo: {{}}",
mensaje_error),
    }
}
```

Nota:

En la función main() anterior, el pimer bloque puede escribirse también de la siguiente forma equivalente:

```
fn main() {
    // Al usar format!, el match devuelve siempre el String
    let resultado_opt = match volumen_cubo(6.0) {
        Some(vol) => format!("¡Éxito! El volumen extraído es {} m³", vol),
        None => format!("Error: El lado indicado no era válido para
    calcular el volumen."),
    };

    // Para comprobación
    println!("{}", resultado_opt);
}
```

Al retornar `format!` en las dos ramas del `match`, este siempre devuelve un string que se almacena en `resultado_opt`. Si en lugar de `format!` utilizamos `println!` entonces no devolvemos nada y la versión anterior no funcionaría.

6.4. El filtro rápido: La estructura `if let`

A veces no te importan los dos lados de la moneda; solo quieres hacer algo si la operación fue bien, ignorando el resto. Para no escribir un `match` engorroso con ramas vacías, usamos este atajo:

```
// Con Option
if let Some(vol) = volumen_cubo(7.0) {
    println!("El volumen es: {}", vol);
}

// Con Result
if let Ok(sup) = superficie_cubo(8.0) {
    println!("La superficie es: {}", sup);
}
```

¿Qué pasa si el lado mide menos de 5.0 y la función devuelve `None` en el primer caso o `Err` en el segundo? Absolutamente nada. El programa simplemente ignora el bloque entre llaves y continúa su camino de forma 100% segura sin colgarse.

6.5. El plan B: El método `unwrap_or()`

¿Y si queremos extraer el valor, pero si algo sale mal preferimos asignar un valor por defecto en lugar de romper el programa? Para eso existe el comodín `unwrap_or()`.

```
// Si el volumen falla (None), nos asigna automáticamente un 0.0 de rescate
let mi_volumen = volumen_cubo(3.0).unwrap_or(0.0);
println!("Volumen final: {}", mi_volumen); // Imprimirá: Volumen final: 0.0
```

6.6. Los botones de autodestrucción: `unwrap()` y `expect()`

Son los métodos más rápidos, pero requieren que estés completamente seguro de lo que haces. Son actos de fe ciega.

`.unwrap()`: Le dice a Rust: «Saca el valor. Si está vacío o es un error, haz que termine el programa».

`.expect("Mensaje de pánico")`: Hace exactamente lo mismo, pero te permite personalizar el mensaje que saldrá en la consola cuando el programa sufra el pánico. Es muy útil para rastrear en qué línea exacta murió tu aplicación.

Ejemplos:

```
let vol = volumen_cubo(10.0).unwrap(); // Seguro, porque 10.0 es mayor que 5.0

// ¡PÁNICO! El programa muere aquí porque 2.0 es inválido, mostrando el mensaje
// personalizado que le pasamos al expect().
let sup = superficie_cubo(2.0).expect("Error fatal al calcular la superficie
del cuaderno");
```

6.7. El pase de oro para profesionales: El operador de propagación ?

Imagina que estás escribiendo una función que hace varios cálculos seguidos y no quieres llenar tu código de infinitos `match` o `if let` anidados. Rust inventó el operador `?` para actuar como un cobrador automático de errores.

Para poder usar el `?`, la función donde lo aplicamos debe devolver el mismo tipo de error (`None` o `Err`) que la función que hace de padre. Su comportamiento es mágico: intenta desempaquetar el valor; si lo logra, te da el número limpio directamente; pero si detecta un error (un `None` o un `Err`), detiene la función en ese mismo instante y «lanza» el error hacia la función padre, delegando en ella la responsabilidad de tratar el error.

Vamos a ver un ejemplo paso a paso.

Definimos abajo una función `informe_cubo()` que en su interior utiliza las dos funciones con las que hemos trabajado anteriormente: `superficie_cubo()` y `volumen_cubo()`.

`informe_cubo()` es la función padre de la que hablamos antes puesto que las dos llamadas con `?` están en su interior.

- En la primera utilización del `?` en `let sup = superficie_cubo(lado)?;`, si `lado` es menor que 5 la función devolverá un `Err(String)` y el operador `?` lo pasará directamente a `informe_cubo()` para que lo trate ya que `informe_cubo()` también devuelve un `Err(String)` y por ello son compatibles. Esta compatibilidad es necesaria para que funcione el `?`.
- En la segunda utilización del `?` en `let vol = volumen_cubo(lado).ok_or("No se pudo calcular el volumen")?;` si `lado` es menor que 5 la función devuelve `None`, por eso hemos añadido `ok_or("No se pudo calcular el volumen")` para que devuelva un `String` en lugar de `None` y sea compatible con la función padre.

```
// Una función que calcula ambas cosas y devuelve un Result combinado
fn informe_cubo(lado: f64) -> Result<String, String> {

    // El '?' extrae el f64 de Ok() si va bien. Si da Err, la función
    // muere AQUÍ y devuelve ese Err.
    let sup = superficie_cubo(lado)?;
    let vol = volumen_cubo(lado).ok_or("No se pudo calcular el volumen")?;
```

```
Ok(format!("Cubo de lado {}: Superficie de {} m² y Volumen de {} m³", lado,
sup, vol))
}
```

Para entender completamente el operador `?` es conveniente que revises el ejercicio que hemos estado comentado pero con la versión entera, incluida la función `main`.

Fichero: **interrogante.rs**

```
fn volumen_cubo(lado: f64) -> Option<f64> {
    if lado < 5.0 {
        // no existe
        return None;
    }

    // resultado válido envuelto en Some()
    let volumen = lado * lado * lado;
    // Como es la última línea, aquí NO se usa 'return' ni punto y coma
    Some(volumen)
}

fn superficie_cubo(lado: f64) -> Result<f64, String> {
    if lado < 5.0 {
        // El return es obligatorio para salir corriendo de la
        // función AQUÍ mismo
        return Err(format!(
            "El lado ({}) es muy pequeño. Mínimo debe ser 5.0",
            lado
        ));
    }

    let superficie = 6.0 * lado * lado;
    // Como es la última línea, aquí NO se usa 'return' ni punto y coma
    Ok(superficie)
}

fn informe_cubo(lado: f64) -> Result<String, String> {
    // El '?' de la línea de abajo extrae el f64 del Ok() de la función
    // superficie_cubo, si va bien.
    // Si da Err, la función muere AQUÍ y devuelve ese Err de superficie_cubo.
    let sup = superficie_cubo(lado)?;
    let vol = volumen_cubo(lado).ok_or("No se pudo calcular el volumen");

    Ok(format!("Cubo de lado {}: Superficie de {} m² y Volumen de {} m³", lado,
sup, vol))
}

fn main() {
    match informe_cubo(3.0){
        Ok(mensaje) => println!("{mensaje}"),
        Err(error) => println!("{error}")
    }
}
```

Cambia el argumento pasado a **informe_cubo**, valores mayores y menores que 5 y observa los resultados.

6.8. Resumen para llevar en la mochila:

- Usa `match` cuando necesites controlar el éxito y el fracaso al mismo nivel.
- Usa `if let` si solo te importa el camino feliz.
- Usa `unwrap_or` si tienes un plan B numérico o un valor por defecto.
- Usa `?` en tus funciones complejas para encadenar operaciones sin ensuciar el código.
- Evita `unwrap` en código de producción, pero aprovéchalo en tus primeros bocetos y pruebas caseras.

7. Limpieza y Orden: Organización en Módulos (`mod`)

Hasta ahora hemos escrito todo nuestro código (estructuras, enumeraciones, métodos y la función `main`) dentro de un único archivo de texto. Para proyectos educativos pequeños funciona bien. Sin embargo, en el mundo real, los programas comerciales tienen miles o millones de líneas de código. Dejarlo todo en un único fichero sería una pesadilla inmanejable.

Rust soluciona esto dividiendo el código en carpetas y archivos independientes a través de su sistema de módulos (`mod`).

7.1. ¿Por qué un solo archivo `main.rs` ya no basta?

Trabajar en un único archivo tiene tres problemas principales:

- **Dificultad de lectura:** Buscar una línea concreta entre miles de líneas es ineficiente.
- **Conflictos de equipo:** Si varios programadores modifican el mismo archivo a la vez, se generan errores al fusionar el trabajo.
- **Mantenimiento:** Mezclar la lógica del motor del juego con la interfaz de usuario hace que arreglar un fallo sea arriesgado.

Organizar el código en módulos nos permite encapsular la información, aislar los problemas y reutilizar componentes fácilmente.

Una forma frecuente de trabajar es dividir el programa total en diversos ficheros, cada uno de los cuales es un módulo. Por ejemplo, podríamos tener en un caso sencillo solo dos archivos, uno con la definición de funciones denominado **funciones.rs** y el archivo principal **main.rs**. Dentro del archivo `main.rs`, la primera instrucción podría ser **`mod funciones`**, para incluir el archivo `funciones.rs` en el archivo principal. Dentro del archivo `funciones.rs`, las funciones se calificarían con la palabra **`pub`** para que sean visibles desde el `main.rs`. Por ejemplo **`pub fn sumar(a:i32, b:i32)`**.

No obstante y con objeto de simplificar, en el siguiente apartado supondremos que creamos módulos dentro de un mismo fichero. Ésta no es una forma habitual de trabajar pero nos permitirá ilustrar de forma sencilla algunas consecuencias del trabajo con módulos.

7.2. Creación de módulos locales y la palabra clave `pub`

Por defecto, **todo en Rust es privado**. Si creas una estructura o una función dentro de un módulo, las partes externas del programa no podrán verla ni usarla a menos que la marques explícitamente como pública utilizando la palabra clave **`pub`**.

Imagina que organizamos nuestro código creando una «caja fuerte» interna (un módulo) dentro del mismo archivo para simular este aislamiento:

 Copia el siguiente código en la Playground y ejecútalo con [RUN]

Fichero: **`main.rs`**

```
// Definimos un módulo interno llamado 'sistema_seguridad'
mod sistema_seguridad {
```

```
// Esta estructura es PÚBLICA porque lleva 'pub'
pub struct TarjetaAcceso {
    pub codigo: u32,
    secreto_interno: u32, // Este campo es PRIVADO (no lleva pub)
}

impl TarjetaAcceso {
    // Constructor público
    pub fn new(codigo: u32, secreto: u32) -> TarjetaAcceso {
        TarjetaAcceso { codigo, secreto_interno: secreto }
    }

    // Método público para validar
    pub fn verificar(&self) -> bool {
        // Un método del módulo sí puede leer sus propios campos privados
        self.secreto_interno > 1000
    }
}

fn main() {
    // Para acceder al contenido del módulo usamos la ruta con
    // los cuatro puntos (::)
    let tarjeta = sistema_seguridad::TarjetaAcceso::new(101, 5555);

    // Esto funciona porque el campo 'codigo' es público:
    println!("Leyendo tarjeta número: {}", tarjeta.codigo);

    // ✗ ERROR DE COMPILACIÓN: Si descomentas la siguiente línea,
    // Rust fallará:
    // println!("El secreto es: {}", tarjeta.secreto_interno);

    if tarjeta.verificar() {
        println!("🔓 Acceso concedido al servidor central.");
    }
}
```

Nota cómo hemos tenido que marcar con **pub** la estructura y los métodos definidos en el **módulo sistema_seguridad** para poder utilizarlos desde el **main()** que no pertenece al módulo.

7.3. Estructura multi-archivo: Separando la lógica en ficheros independientes

El verdadero poder llega cuando extraemos el módulo a su propio fichero físico en el disco duro. En las versiones modernas de Rust, esto es sorprendentemente limpio.

Imagina que en tu proyecto tienes dos archivos dentro de la carpeta **src/**:

1. **src/autenticacion.rs** (El archivo donde guardaremos la lógica de validación).
2. **src/main.rs** (El archivo principal que coordina la aplicación).

Veamos los listados de estos dos archivos:

Fichero 1: **src/autenticacion.rs**

Este archivo va a constituir el **módulo autenticacion**. En él no ponemos la palabra clave `mod`. Escribimos directamente las estructuras y funciones que queremos empaquetar, asegurándonos de usar **pub** para que sean visibles desde otro archivo, concretamente desde `main.rs`.

```
// src/autenticacion.rs

pub struct Usuario {
    pub nombre_usuario: String,
    email: String, // Privado: controlamos su acceso
}

impl Usuario {
    pub fn new(nombre: &str, correo: &str) -> Usuario {
        Usuario {
            nombre_usuario: String::from(nombre),
            email: String::from(correo),
        }
    }

    // Método público para verificar si el correo es institucional:
    // contiene la @ y termina en .com
    pub fn tiene_email_valido(&self) -> bool {
        self.email.contains('@') && self.email.ends_with(".com")
    }
}
```

Fichero 2: `src/main.rs`

Para enlazar el archivo externo, usamos la palabra clave **mod** al principio de nuestro archivo principal. Rust buscará automáticamente un fichero que se llame igual que el módulo.

```
// src/main.rs

// 1. Le decimos a Rust que busque y cargue el archivo 'autenticacion.rs'
mod autenticacion;

// Opcional: usamos 'use' para crear un atajo y no escribir la
// ruta completa cada vez
// Podremos escribir 'Usuario::new' en lugar de 'autenticacion::Usuario::new'
use autenticacion::Usuario;

fn main() {
    println!("=== 🛡️ SISTEMA DE ALTA DE USUARIOS ===");

    // 2. Instanciamos el objeto usando el módulo externo
    let nuevo_perfil = Usuario::new("Halcón_Retro", "halcon68@correo.com");

    // 3. Ejecutamos la lógica aislada
    if nuevo_perfil.tiene_email_valido() {
        println!("✅ Registro completado. Bienvenido, {}. ",
nuevo_perfil.nombre_usuario);
    } else {
        println!("❌ Error: El correo electrónico no cumple con los
requisitos.");
    }
}
```

```
}
}
```

 Vamos a escribir los archivos anteriores en un proyecto Rust con Cargo.

Proyecto: **mod_autenticacion**

1. Abre una terminal integrada de Visual Studio Code en la carpeta **proyectos-rust** que creaste al principio o en cualquier otra. Ejecuta la orden **cargo new mod_autenticacion**. Se creará la carpeta de proyecto `mod_autenticacion` con varios ficheros y el directorio **src**. En dicho directorio se encuentra el fichero **main.rs**. Haz clic derecho sobre la carpeta **src** y agrega un nuevo fichero al directorio con el nombre **autenticacion.rs**. Justo arriba de este texto tienes indicado el código que tienes que añadir a ambos ficheros, `autenticacion.rs` y `main.rs`.
2. Abre una terminal en la carpeta del proyecto o ejecuta **cd mod_autenticacion** y ejecuta la orden **cargo run** para compilar y ejecutar el programa.

Observa que esta vez, al tener el módulo externo `autenticacion.rs` hemos tenido que incluirlo en el `main.rs` usando `mod`.

7.4. Proyecto Práctico III: Validador robusto de datos de usuario en producción

Para consolidar este cuaderno, deberás simular un sistema profesional dividiendo el código. Crearemos una estructura que valide las credenciales de un registro de usuario utilizando módulos y devolviendo resultados controlados mediante enumeraciones `Option`.

Si juntáramos la arquitectura multi-archivo en un bloque lógico simulado (para entender su interacción completa), el motor de registro luciría así:

```
// Simulación del módulo 'validador' que estaría en otro archivo
mod validador {
    pub struct Registro {
        usuario: String,
        clave: String,
    }

    impl Registro {
        pub fn new(usuario: &str, clave: &str) -> Registro {
            Registro {
                usuario: String::from(usuario),
                clave: String::from(clave),
            }
        }

        // Analiza las reglas del sistema y devuelve un Option
        // Si hay un error, devuelve Some(Mensaje).
        // Si todo está limpio, devuelve None.
        // Funciona con la lógica cambiada.
        // Utiliza Some para indicar error y None cuando todo está bien
        pub fn procesar_seguridad(&self) -> Option<String> {
            if self.usuario.len() < 4 {
```

```

        return Some(String::from("El nombre de usuario debe tener al
menos 4 caracteres."));
    }
    if self.clave.len() < 6 {
        return Some(String::from("La contraseña es demasiado corta e
insegura (mínimo 6)."));
    }
    if self.clave == self.usuario {
        return Some(String::from("La contraseña no puede ser idéntica
al nombre de usuario."));
    }

    // Si no se ha producido ninguno de los errores anteriores,
    // mandamos None para indicar que no hay mensajes de error:
    // Perfecto, sin errores
    None
}
}

fn main() {
    println!("--- FORMULARIO DE INSCRIPCIÓN ---");

    // Caso de prueba 1: Intento de registro erróneo
    let intento_uno = validador::Registro::new("H68", "123");

    // Evaluamos el resultado usando el control de flujo match sobre el Option
    match intento_uno.procesar_seguridad() {
        Some(error) => println!("❌ Registro denegado: {}", error),
        None => println!("🎉 ¡Cuenta creada con éxito!"),
    }

    // Caso de prueba 2: Intento de registro correcto
    let intento_dos = validador::Registro::new("FalconRustaceo",
"ClaveSegura2026");

    match intento_dos.procesar_seguridad() {
        Some(error) => println!("❌ Registro denegado: {}", error),
        None => println!("🎉 ¡Cuenta creada con éxito! El sistema está
listo."),
    }
}

```

📌 **Nota:** Ten en cuenta que la función `.procesar_seguridad` devuelve un `Option<string>`. En el `main`, desempaquetamos el `Option` con un `match` y según lo que tenga dentro imprimimos un mensaje u otro.

1. Crea un proyecto de Rust siguiendo los pasos del apartado anterior. Utiliza la orden **cargo new validador**.
2. En la carpeta **src** crea el fichero **validador.rs** al lado del fichero existente **main.rs**.

3. Distribuye convenientemente el código que tienes arriba entre los dos ficheros, utilizando correctamente la palabra **mod**. La solución la tienes en la carpeta **código > cuaderno3** del repositorio con el nombre de proyecto **validador**.

8. Ejercicios Resueltos del Cuaderno 3

8.1. ENUNCIADOS

8.1.1. Ejercicio 1: Crea tu propia Struct y Función Asociada (Constructor)

Enunciado: Crea una estructura llamada **Dimensiones2D** que contenga los campos **ancho** y **alto** (ambos `f32`). Implementa un bloque **impl** que contenga una función asociada llamada **cuadrado** que reciba un solo parámetro (el lado) y devuelva una instancia de la estructura donde el ancho y el alto sean iguales. Pruébalo en el main.

8.1.2. Ejercicio 2: Métodos Mutables (&mut self)

Enunciado: Define una estructura **Contador** que guarde un valor entero **u32**. Crea un **método mutable** llamado **incrementar** que sume 1 al valor interno, y otro método de lectura llamado **obtener_valor** que devuelva dicho número. Simula un par de incrementos en el main.

8.1.3. Ejercicio 3: Enumeraciones con Datos Asociados

Enunciado: Diseña una enumeración llamada **Dispositivo** que represente elementos de una red informática. Tendrá dos variantes: **Ordenador** (que guarda un String con el nombre de usuario) y **Router** (que guarda una tupla con 4 números `u8` para simular su dirección IP). Instancia uno de cada tipo en el main.

8.1.4. Ejercicio 4: Extrayendo Datos con match

Enunciado: Utilizando el enumerado **Dispositivo** del ejercicio anterior, escribe una función que **reciba dicho enumerado** y, mediante un **bloque match**, imprima un mensaje personalizado. Si es un ordenador, debe decir a quién pertenece; si es un router, debe mostrar su dirección IP formateada con puntos.

8.1.5. Ejercicio 5: El patrón por defecto _ e if let con Option

Enunciado: En los videojuegos, la tasa de críticos multiplica el daño. Crea una variable de tipo `Option<u32>` que represente el daño crítico extra. Si tiene valor (Some), imprímelo en pantalla. Si no tiene valor (None), usa un **bloque if let** para capturar de forma directa el estado y mostrar un mensaje avisando que ha sido un ataque normal.

8.2. SOLUCIONES

8.2.1. Ejercicio 1: Crea tu propia Struct y Función Asociada

```
struct Dimensiones2D {
    ancho: f32,
    alto: f32,
}

impl Dimensiones2D {
    // Función asociada que actúa como constructor específico para cuadrados
    fn cuadrado(lado: f32) -> Dimensiones2D {
        Dimensiones2D {
            ancho: lado,
            alto: lado,
        }
    }
}
```

```

    }
}

fn main() {
    let lienzo = Dimensiones2D::cuadrado(4.5);
    println!("Lienzo creado: {}m de ancho por {}m de alto.", lienzo.ancho,
    lienzo.alto);
}

```

Explicación: Las funciones asociadas no reciben `self` porque no operan sobre un objeto ya existente. En este caso, funciona como un constructor alternativo muy útil cuando queremos inicializar una estructura con reglas específicas de forma limpia.

8.2.2. Ejercicio 2: Métodos Mutables (`&mut self`)

```

struct Contador {
    valor: u32,
}

impl Contador {
    // Método mutable: necesita modificar el estado interno
    fn incrementar(&mut self) {
        self.valor += 1;
    }

    // Método de lectura: solo lee el valor
    fn obtener_valor(&self) -> u32 {
        self.valor
    }
}

fn main() {
    // La instancia DEBE ser mutable para poder usar &mut self
    let mut mi_contador = Contador { valor: 0 };

    mi_contador.incrementar();
    mi_contador.incrementar();

    println!("El valor del contador es: {}", mi_contador.obtener_valor());
}

```

Explicación: En Rust, si un método necesita alterar los datos que están dentro de la estructura, debe declararse explícitamente con un parámetro `&mut self`. Además, cualquier variable (ej: `mi_contador`) que use dicho método en el `main` debe ser declarada obligatoriamente con la palabra clave `mut`.

8.2.3. Ejercicio 3: Enumeraciones con Datos Asociados

```

enum Dispositivo {
    Ordenador(String),
    Router(u8, u8, u8, u8),
}

```

```
fn main() {
    let pc_alumno = Dispositivo::Ordenador(String::from("Lucas"));
    let router_aula = Dispositivo::Router(192, 168, 1, 1);

    println!("Dispositivos configurados en la memoria correctamente.");
}
```

Explicación: A diferencia de las enumeraciones clásicas de otros lenguajes, Rust nos permite adjuntar información heterogénea a cada variante. El tipo **Dispositivo** puede contener tanto **una cadena de texto** como **una tupla de cuatro bytes** según la variante que esté activa.

8.2.4. Ejercicio 4: Extrayendo Datos con match

```
enum Dispositivo {
    Ordenador(String),
    Router(u8, u8, u8, u8),
}

fn identificar_nodo(aparato: Dispositivo) {
    match aparato {
        Dispositivo::Ordenador(usuario) => {
            println!("💻 Estación de trabajo activa. Usuario actual: {}",
usuario);
        }
        Dispositivo::Router(a, b, c, d) => {
            println!("🌐 Puerta de enlace detectada. IP: {}.{}.{}.{}", a, b, c,
d);
        }
    }
}

fn main() {
    let pc = Dispositivo::Ordenador("Juan".to_string());
    let nodo = Dispositivo::Router(10, 0, 0, 254);

    identificar_nodo(pc);
    identificar_nodo(nodo);
}
```

Explicación: El bloque match no solo comprueba qué variante del enumerado se está ejecutando, sino que realiza un proceso llamado desestructuración: «abre» la variante y extrae las variables internas (usuario o los octetos a, b, c, d) para que podamos operar con ellas dentro de las llaves correspondientes.

8.2.5. Ejercicio 5: El patrón por defecto _ e if let con option

```
fn main() {
    // 1. El jugador ha tenido mala suerte y no ha obtenido bonificador crítico
    let daño_critico: Option<u32> = None;

    // Usamos 'if let' para buscar únicamente el caso de éxito (Some)
    if let Some(bono) = daño_critico {
```

```

        println!("🔥 ¡Golpe Crítico! Daño extra: +{}", bono);
    } else {
        // El bloque else actúa como el patrón por defecto para capturar el
None
        println!("🔪 Ataque normal. No se aplicaron multiplicadores.");
    }

    // 2. El jugador tiene ahora buena suerte y obtiene bono crítico de 50
    let daño_crítico: Option<u32> = Some(50);

    // Usamos 'if let' para buscar únicamente el caso de éxito (Some)
    if let Some(bono) = daño_crítico {
        println!("🔥 ¡Golpe Crítico! Daño extra: +{}", bono);
    } else {
        // El bloque else actúa como el patrón por defecto para capturar el
None
        println!("🔪 Ataque normal. No se aplicaron multiplicadores.");
    }
}

```

Explicación: El atajo **if let** reduce enormemente las líneas de código cuando **solo nos interesa reaccionar ante una variante específica de una enumeración** (normalmente `Some`), relegando todas las demás opciones posibles a un **bloque else** general y opcional sin necesidad de escribir un `match` exhaustivo. Con **if let** no utilizamos el patrón por defecto `_` que queda sustituido por el bloque **else** si queremos utilizarlo.

Nota también que hemos utilizado dos veces la definición de variable **let daño_crítico**. Esto se denomina en Rust **Shadowing** (sombreado) y te ahorra el tener que inventar un nombre distinto para la variable. Muy pocos lenguajes permiten hacer este sombreado de variable.